

sno.	python	java	meaning
1	pip install <package>	Add <dependency> in pom.xml then run: mvn install	Install a new package/dependency
2	pyproject.toml- it is pom.xml but dependencies/packages/plugins will only have version range and not exactly the version. poetry.lock - poetry is like maven a project management/build tool. Poetry.lock will be autogenerated by poetry, it will have the exact version of all the dependices/packages/plugins present in pyproject.toml, the exact version is generated using an algorithm which determines the latest stable version based on the range specified in pyproject.toml. So without poetry.lock for the same pyproject.toml in different machine different poetry.lock files will be auto generated causing dependencies mismatch. The solution to the mismatch problem is simply to commit the auto-generated poetry.lock to Git. Commit poetry.lock to Git ,Everyone runs: poetry install,Poetry reads	pom.xml--> it has list of configurations, dependencies, details about project, plugins,.. maven is a build tool, it reads the dependencies/plugins from pom.xml and searches it in local .m2 repo(folder --/.m2/repository/ (shared globally)) if not available then it tries to download from central maven repo (Maven Central (https://repo.maven.apache.org)) if not there also then from any user defined repo and place it it .m2 repo .Also it places the downloaded dependencies/plugins in the classpath for the JVM to use. ----- ~/.m2/repository/ (local shared global repo)	Build project

poetry.lock and installs the EXACT same versions everywhere. poetry.lock is always auto-generated. You only commit it to Git. Never create or edit it manually. ----- poetry reads the dependencies/plugins from poetry.lock and searches it in local repo, there are 2 local repos, 1. <code>~/.cache/pypoetry/cache/</code> (global local repo) 2. <code>.venv/lib/site-packages/</code> (per project local repo) first checks in per project local repo (<code>.venv/lib/site-packages/</code>) if not available then it tries to copy/install from global local repo (<code>~/.cache/pypoetry/cache/</code>) if not available then central poetry repo(PyPI (https://pypi.org)) if not there also then from any user defined repo and places it in <code>.venv/lib/site-packages/</code> (per project local repo) and global local repo, so pvm can use it from per project local repo (<code>.venv/lib/site-packages/</code>) ----- <code>.venv/lib/site-packages/</code> (destination, never a source		
---	--	--

	to download from.)		
3	in python after poetry run- Executes your Python script inside the project's .venv. .pyc files are created inside __pycache__ folder, __pycache__ folder will be generated by CPython interpreter. Python can run raw .py source code directly; CPython handles bytecode compilation automatically at execution time.	in java after maven compile-Invokes javac to build .class files in target/. .class files are created inside target folder, target folder will be generated by maven itself. Java cannot run raw .java source code; it must be compiled to .class files first.	Compilation of source code

Step	Java (Maven)	Python (Poetry)
Config/Manifest File	pom.xml	pyproject.toml
Exact Resolved Versions	Handled in pom.xml / internal resolution	poetry.lock
Check 1: Project Installed Path	Checked directly via target/Classpath	.venv/lib/site-packages/
Check 2: Global Local Cache	~/.m2/repository/	~/.cache/pypoetry/cache/

Check 3: Remote Repository	Maven Central (repo.maven.apache.org)	PyPI (pypi.org)
Check 4: Custom Repository	User-defined <repository>	User-defined [[tool.poetry.source]]
Execution Engine	JVM	PVM